Projekt „Kaffeeshop“: Vorbereitung für Fr 13.11.

Grundlage: Folien „vl07_projekt_onlineshop“ (Folien 1–23) und das Startpaket `VL07_Startpaket.zip` aus Moodle. Alle Code-Stücke hier sind eigene Lösungen. Sie sind mit Python 3.13 ausgeführt und mit `ruff check` und `ty check` geprüft (beide melden nichts). Die Ausgaben stimmen mit den Kontrollwerten der Folien überein.

---

1. Was ist das Projekt?

In einem Satz: Ihr programmiert zu zweit einen kleinen Onlineshop für Kaffee und Zubehör. Er läuft im Terminal: Man sieht das Sortiment, sucht Produkte, legt sie in den Warenkorb und bestellt.

Stell dir vor: Du bist auf einer Shop-Website. Links ist das Regal mit allen Produkten, das ist das **Lager**. Rechts ist dein **Warenkorb**. Jede Zeile im Warenkorb („2 x Kaffee, 30,92 €“) ist eine **Position**. Oben steht dein Name, das ist der **Kunde**. Genau diese vier Dinge baut ihr als Klassen nach, nur ohne Bilder und mit Menü im Terminal.

Wie der Tag abläuft (Folie 4):

Schritt	Was passiert
Einstieg	Frage zur Geräteverwaltung aus VL06 (letzter Termin)
Aufgabe + Startpaket	Zip aus Moodle laden, in `prog1/kaffeeshop` entpacken
Stufe 1 (Pflicht)	Lager, Warenkorb, Bestellung
Zwischenstand	Zwei Teams zeigen ihr Programm
Stufe 2	Produktarten (7 % / 19 % MwSt) und Zahlungsarten
Stufe 3	Zusatz nach Wahl (Versand, Rabattcode, Sortieren, …)
Abschluss	Einige Teams stellen vor, danach Musterlösung in Moodle

Regeln (Folie 5):

- Zu zweit an **einem** Rechner. Eine Person tippt, die andere denkt mit, Wechsel etwa alle 30 Minuten.
- **Beide müssen jede Zeile erklären können.** Darum steht hier zu jedem Teil „Auf Deutsch heißt das“.
- Editor **ohne KI**. Geprüft wird mit `uv run ruff check .` und `uv run ty check`.
- **Ziel:** Am Ende läuft bei jedem Team mindestens Stufe 1.

> **So nutzt du diese Vorbereitung:** Lies sie heute einmal durch, damit du morgen weißt, wohin es geht. Tipp den Code morgen aber selbst, mit deinem Partner. Nicht kopieren. Wer den Code einmal verstanden hat, schreibt ihn in 15 Minuten selbst, und genau dieses Schreiben ist die Übung für die Klausur.

---

2. Was im Startpaket schon fertig ist

Datei	Inhalt	Musst du ändern?
`produkt.py`	Klasse `Produkt`: `nummer`, `name`, `preis` (netto), `bestand`, `bruttopreis` (+19 %), `einlagern()`, `verkaufen()`	Stufe 1: nein. Stufe 2: ja
`daten.py`	`beispielsortiment()`: 8 Produkte, 101–104 Lebensmittel, 201–204 Zubehör	Stufe 1: nein. Stufe 2: ja
`main.py`	Menü mit `while True` + `match`. Punkt 1 und 0 gehen. Fertige Funktion `zahl_einlesen()`	ja

Neu schreibst du: `lager.py` und `warenkorb.py`. In Stufe 2 kommen `zahlungsarten.py` und `bestellung.py` dazu.

Drei Dinge aus `produkt.py`, die du kennen musst:

- `produkt.bruttopreis` ist eine **Property**. Du schreibst sie **ohne Klammern**, und sie liefert den Preis mit Steuer, gerundet auf Cent. Kaffee: 12,99 · 1,19 = 15,4581 → **15.46**.
- `produkt.verkaufen(menge)` verringert den Bestand. Ist zu wenig da, gibt sie `False` zurück und ändert nichts.
- `zahl_einlesen("Frage: ")` fragt so lange nach, bis nur Ziffern kommen. Bei `abc` stürzt das Programm also nicht ab.

---

3. Das Bild: Wer hat was?

In Stufe 1 gibt es **keine Vererbung**, nur **Hat-ein-Beziehungen**. Ein Objekt speichert andere Objekte als Attribut.

Klasse	Hat	Kann (Methoden)	Datei
`Lager`	viele `Produkt` (Liste `produkte`)	`anzeigen()`, `finden(nummer)`, `suchen(text)`	`lager.py`
`Kunde`	`name`, `email`	– (nur Daten, `@dataclass`)	`warenkorb.py`
`Position`	ein `Produkt` + `menge`	`summe()`	`warenkorb.py`
`Warenkorb`	einen `Kunde` + viele `Position`	`hinzufuegen`, `entfernen`, `gesamtsumme`, `ist_leer`, `anzeigen`	`warenkorb.py`

Merksatz: Lager und Warenkorb sind beide „Listen mit Extras“. Fast jede Methode ist eins von drei Schleifen-Mustern, die du schon kennst:

Muster	Frage	Beispiel im Projekt
Finden	„Gibt es das eine?“ → sofort `return`, nach der Schleife `return None`	`Lager.finden`, `Warenkorb.position_finden`
Sammeln	„Welche passen alle?“ → leere Liste, `append` in der Schleife	`Lager.suchen`
Aufsummieren	„Wie viel zusammen?“ → `summe = 0`, in der Schleife `+=`	`Warenkorb.gesamtsumme`

---

4. Stufe 1 Schritt für Schritt

4.1 `lager.py`: das Lager (Folie 10)

```
"""Projekt Kaffeeshop, Stufe 1: das Lager."""

from produkt import Produkt


class Lager:
    def __init__(self, produkte: list[Produkt]) -> None:
        self.produkte = produkte  # Lager HAT viele Produkte

    def anzeigen(self) -> None:
        for produkt in self.produkte:
            print(produkt)

    def finden(self, nummer: int) -> Produkt | None:
        """Das Produkt mit dieser Artikelnummer, sonst None."""
        for produkt in self.produkte:
            if produkt.nummer == nummer:
                return produkt  # gefunden: sofort raus
        return None  # Schleife ganz durch, nichts gefunden

    def suchen(self, text: str) -> list[Produkt]:
        """Alle Produkte, deren Name den Text enthält (groß/klein egal)."""
        treffer: list[Produkt] = []
        for produkt in self.produkte:
            if text.lower() in produkt.name.lower():
                treffer.append(produkt)
        return treffer
```

Auf Deutsch heißt das:

- `__init__` merkt sich die Produktliste. Mehr braucht das Lager nicht.
- `anzeigen` geht jedes Produkt durch und druckt es. `print(produkt)` ruft dabei `__str__` aus `produkt.py` auf, daher kommt die Zeile „101 Kaffee Hausmischung 1 kg: 15.46 Euro (40 auf Lager)“.
- `finden` ist das **Finden-Muster**. Das `return None` steht **nach** der Schleife, also nicht eingerückt unter dem `if`. Sonst würde schon das erste nicht passende Produkt „nichts gefunden“ melden.
- `-> Produkt | None` heißt: Die Methode liefert ein Produkt **oder** `None`.
- `suchen` ist das **Sammeln-Muster**. `"kaffee" in "kaffee hausmischung 1 kg"` ist `True`. Weil beide Seiten `.lower()` bekommen, ist Groß- und Kleinschreibung egal.

Rundentabelle `suchen("kaffee")`:

Runde	`produkt.name.lower()`	enthält „kaffee“?	`treffer` danach
1	kaffee hausmischung 1 kg	ja	[101]
2	espresso bohnen 500 g	nein	[101]
3–4	grüner tee …, kakaopulver …	nein	[101]
5	kaffeemühle mit handkurbel	ja	[101, 201]
6–8	filterpapier …, french press …, thermobecher …	nein	[101, 201]

Ergebnis: **zwei Treffer**, genau wie Folie 14 es verlangt.

4.2 `warenkorb.py`: Kunde, Position, Warenkorb (Folie 12)

```
"""Projekt Kaffeeshop, Stufe 1: Kunde, Position und Warenkorb."""

from dataclasses import dataclass
from decimal import Decimal

from produkt import Produkt


@dataclass
class Kunde:
    name: str
    email: str


@dataclass
class Position:
    produkt: Produkt  # Position HAT ein Produkt
    menge: int

    def summe(self) -> Decimal:
        return self.produkt.bruttopreis * self.menge
```

Auf Deutsch heißt das:

- `@dataclass` schreibt `__init__` für dich. `Kunde("Anna Berger", "anna@example.com")` funktioniert sofort. Das kennst du aus VL05, Übung 5.
- `Position.summe()` rechnet Bruttopreis mal Menge: 2 · 15,46 = **30.92**.

Jetzt der Warenkorb selbst:

```
class Warenkorb:
    def __init__(self, kunde: Kunde) -> None:
        self.kunde = kunde  # Warenkorb HAT einen Kunden
        self.positionen: list[Position] = []  # ... und viele Positionen

    def position_finden(self, nummer: int) -> Position | None:
        for position in self.positionen:
            if position.produkt.nummer == nummer:
                return position
        return None

    def menge_im_korb(self, nummer: int) -> int:
        position = self.position_finden(nummer)
        if position is None:
            return 0
        return position.menge

    def hinzufuegen(self, produkt: Produkt, menge: int) -> bool:
        """False, wenn die Menge (mit dem, was schon im Korb liegt) nicht auf Lager ist."""
        if menge <= 0:
            return False
        if self.menge_im_korb(produkt.nummer) + menge > produkt.bestand:
            return False
        position = self.position_finden(produkt.nummer)
        if position is None:
            self.positionen.append(Position(produkt, menge))  # neue Position
        else:
            position.menge += menge  # schon im Korb: nur Menge erhöhen
        return True

    def entfernen(self, nummer: int) -> bool:
```

```
        position = self.position_finden(nummer)
        if position is None:
            return False
        self.positionen.remove(position)
        return True

    def gesamtsumme(self) -> Decimal:
        summe = Decimal(0)
        for position in self.positionen:
            summe += position.summe()
        return summe

    def ist_leer(self) -> bool:
        return len(self.positionen) == 0

    def anzeigen(self) -> None:
        print(f"Warenkorb von {self.kunde.name}")
        if self.ist_leer():
            print("Der Warenkorb ist leer.")
            return
        for position in self.positionen:
            print(f"{position.menge} x {position.produkt.name}: {position.summe()} Euro")
        print(f"Summe: {self.gesamtsumme()} Euro")
```

Auf Deutsch heißt das:

- `position_finden` und `menge_im_korb` sind zwei kleine **Hilfsmethoden**. Die Folie verlangt sie nicht, aber mit ihnen werden `hinzufuegen` und `entfernen` kurz. Außerdem braucht `main.py` für die Meldung „schon im Warenkorb: 3“ genau diese Zahl.
- **`hinzufuegen` ist die wichtigste Methode des Tages.** Sie läuft in drei Schritten ab:
  1. Ist die Menge unsinnig (0 oder weniger)? Dann `False`.
  2. **Was schon im Korb liegt, zählt mit.** Liegen 3 French Press im Korb und es sind 3 auf Lager, ist 3 + 1 = 4 > 3. Dann `False`.
  3. Liegt das Produkt schon im Korb, wird nur die **Menge erhöht**. Sonst kommt eine **neue Position** dazu. So ergibt zweimal Kaffee **eine** Position und nicht zwei.
- `entfernen` sucht die Position und nimmt sie mit `list.remove()` aus der Liste.
- `gesamtsumme` ist das **Aufsummieren-Muster**. Es startet bei `Decimal(0)`, nicht bei `0`, damit `ty` zufrieden ist.
- `ist_leer` gibt direkt den Vergleich zurück. Ein `if … return True else return False` ist dafür unnötig.

Durchgespielt: French Press (Bestand 3)

Aufruf	im Korb vorher	Prüfung	Ergebnis	im Korb nachher
`hinzufuegen(fp, 5)`	0	0 + 5 > 3 → zu viel	`False`	0
`hinzufuegen(fp, 3)`	0	0 + 3 > 3? nein	`True` (neue Position)	3
`hinzufuegen(fp, 1)`	3	3 + 1 > 3 → zu viel	`False`	3

4.3 `main.py`: das Menü fertig bauen (Folien 11 und 13)

Oben in `main.py` kommen zwei Imports dazu:

```
from lager import Lager
from warenkorb import Kunde, Warenkorb
```

`MENUE` und `zahl_einlesen()` bleiben, wie sie sind. Darunter kommt **für jeden Menüpunkt eine Funktion** (Tipp der Folie 11):

```
def suchen(lager: Lager) -> None:
    treffer = lager.suchen(input("Suchbegriff: "))
    if len(treffer) == 0:
        print("Kein Produkt gefunden.")
    for produkt in treffer:
        print(produkt)


def in_den_warenkorb(lager: Lager, korb: Warenkorb) -> None:
    nummer = zahl_einlesen("Artikelnummer: ")
    produkt = lager.finden(nummer)
    if produkt is None:
        print(f"Ein Produkt mit der Nummer {nummer} gibt es nicht.")
        return
    menge = zahl_einlesen("Menge: ")
    if menge == 0:
        print("Die Menge muss mindestens 1 sein.")
        return
    if korb.hinzufuegen(produkt, menge):
        print(f"{menge} x {produkt.name} liegt im Warenkorb.")
    else:
        schon_drin = korb.menge_im_korb(nummer)
        print(
            f"Das geht nicht. Auf Lager: {produkt.bestand}, "
            f"schon im Warenkorb: {schon_drin}."
        )


def aus_dem_warenkorb(korb: Warenkorb) -> None:
    nummer = zahl_einlesen("Artikelnummer: ")
    if korb.entfernen(nummer):
        print("Entfernt.")
    else:
        print(f"Die Nummer {nummer} liegt nicht im Warenkorb.")


def bestellen(korb: Warenkorb) -> None:
    if korb.ist_leer():
        print("Der Warenkorb ist leer. Es gibt nichts zu bestellen.")
        return
    summe = korb.gesamtsumme()  # vor dem Leeren merken
    for position in korb.positionen:
        position.produkt.verkaufen(position.menge)  # Bestand verringern
    korb.positionen = []  # neue leere Liste
    print(f"Vielen Dank, {korb.kunde.name}! Ihre Bestellung über {summe} Euro.")


def main() -> None:
    lager = Lager(beispielsortiment())
    print("Willkommen im Kaffeeshop!")
    kunde = Kunde(input("Ihr Name: "), input("Ihre E-Mail: "))
    korb = Warenkorb(kunde)
    while True:
        print(MENUE)
        match input("Ihre Wahl: "):
            case "1":
                lager.anzeigen()
            case "2":
                suchen(lager)
            case "3":
                in_den_warenkorb(lager, korb)
            case "4":
```

```
                korb.anzeigen()
            case "5":
                aus_dem_warenkorb(korb)
            case "6":
                bestellen(korb)
            case "0":
                print("Auf Wiedersehen!")
                break
            case _:
                print("Bitte eine Zahl aus dem Menü eingeben.")
```

Auf Deutsch heißt das:

- Jede Funktion bekommt **als Parameter die Objekte, die sie braucht**: `suchen` braucht nur das Lager, `in_den_warenkorb` braucht Lager und Korb. Dadurch bleibt `main()` kurz.
- `if produkt is None: … return` ist ein **früher Ausstieg**: Bei einem Fehler wird eine Meldung ausgegeben und die Funktion beendet. Der Rest der Funktion muss sich dann nicht mehr um diesen Fall kümmern.
- In `bestellen` wird die Summe **vor** dem Leeren gemerkt. Danach ist der Korb leer und die Summe wäre 0.
- `case _:` ist der „alles andere"-Fall von `match`, zum Beispiel bei der Eingabe `x`.

4.4 Testen (Folie 14): So sieht es richtig aus

Test	Eingabe	Erwartete Ausgabe
Suche	2, `kaffee`	101 Kaffee Hausmischung … 15.46 Euro und 201 Kaffeemühle … 34.99 Euro
Zu viel	3, `203`, `5`	Das geht nicht. Auf Lager: 3, schon im Warenkorb: 0.
Erst 3, dann 1	3, `203`, `3` → 3, `203`, `1`	erst „3 x French Press 1 l liegt im Warenkorb.", dann „… schon im Warenkorb: 3."
Zweimal Kaffee	3, `101`, `2` → 3, `101`, `1`	Menüpunkt 4 zeigt **eine** Zeile: 3 x Kaffee Hausmischung 1 kg: 46.38 Euro
Unbekannt	3, `999`	Ein Produkt mit der Nummer 999 gibt es nicht.
Keine Zahl	3, `abc`	Bitte eine ganze Zahl eingeben. (kein Absturz)
Bestellen	6	Vielen Dank, … Ihre Bestellung über … Euro. Danach: Korb leer, Bestand kleiner (Kaffee 40 → 37)
Leer bestellen	6 bei leerem Korb	Der Warenkorb ist leer. Es gibt nichts zu bestellen.
Prüfwerkzeuge	`uv run ruff check .` und `uv run ty check`	All checks passed!

---

5. Stufe 2: Produktarten und Zahlungsarten (Folien 16–19)

5.1 Produktarten: 7 % oder 19 % MwSt

In einem Satz: Lebensmittel wie Kaffee und Tee kosten 7 % Steuer, Zubehör 19 %. Jede Produktart soll ihren Satz selbst kennen.

Das ist Polymorphie aus VL06: `bruttopreis` ruft `self.mwst_satz()` auf. Welcher Satz dabei herauskommt, hängt davon ab, ob das Objekt ein `Lebensmittel` oder ein `Zubehoer` ist.

Änderungen in **`produkt.py`**:

1. `from abc import ABC, abstractmethod` und `from typing import override` importieren. Die Konstante `MWST_SATZ` fällt weg.
2. `class Produkt(ABC):` macht `Produkt` abstrakt.
3. In `bruttopreis` wird `MWST_SATZ` durch `self.mwst_satz()` ersetzt: `return (self._preis * (1 + self.mwst_satz())).quantize(Decimal("0.01"))`
4. Die abstrakte Methode kommt in die Klasse `Produkt`, und unten in der Datei kommen die zwei Unterklassen dazu:

```
    @abstractmethod
    def mwst_satz(self) -> Decimal:
        """Jede Produktart legt ihren Steuersatz selbst fest."""


class Lebensmittel(Produkt):
    @override
    def mwst_satz(self) -> Decimal:
        return Decimal("0.07")  # ermäßigter Satz, auch für Kaffee und Tee


class Zubehoer(Produkt):
    @override
    def mwst_satz(self) -> Decimal:
        return Decimal("0.19")
```

5. Optional, aber sauber wie in den VL06-Musterlösungen: `@override` über `def __str__`, weil `__str__` eine Methode von `object` überschreibt. `ty` meldet ohne es nichts.

Änderungen in **`daten.py`**: `from produkt import Lebensmittel, Zubehoer`. Dann wird aus `Produkt(101, …)` bis `Produkt(104, …)` jeweils `Lebensmittel(…)`, und aus `Produkt(201, …)` bis `Produkt(204, …)` jeweils `Zubehoer(…)`.

Kontrolle (Folie 16): Kaffee 12,99 · 1,07 = 13,8993 → **13.90** (vorher 15.46). French Press 20,90 · 1,19 = 24,871 → **24.87** (bleibt gleich).

5.2 Zahlungsarten: `zahlungsarten.py`

```
"""Projekt Kaffeeshop, Stufe 2: Zahlungsarten (wie in VL06)."""

from abc import ABC, abstractmethod
from decimal import Decimal
from typing import override


class Zahlungsart(ABC):
    @abstractmethod
    def bezahlen(self, betrag: Decimal) -> str:
        """Jede Zahlungsart bezahlt auf ihre Weise und liefert einen Text."""


class Kreditkarte(Zahlungsart):
    def __init__(self, nummer: str) -> None:
        self.nummer = nummer

    @override
    def bezahlen(self, betrag: Decimal) -> str:
        return f"{betrag} Euro von Karte ...{self.nummer[-4:]} abgebucht"
```

```
class Lastschrift(Zahlungsart):
    def __init__(self, iban: str) -> None:
        self.iban = iban

    @override
    def bezahlen(self, betrag: Decimal) -> str:
        return f"{betrag} Euro per Lastschrift vom Konto ...{self.iban[-4:]}"


class Rechnungskauf(Zahlungsart):
    @override
    def bezahlen(self, betrag: Decimal) -> str:
        return f"Rechnung über {betrag} Euro, zahlbar in 14 Tagen"
```

Auf Deutsch heißt das: `self.nummer[-4:]` sind die letzten vier Zeichen, also „5678“. `Zahlungsart()` selbst kann man nicht anlegen, weil sie abstrakt ist. Das gibt einen `TypeError`.

5.3 Die Bestellung: `bestellung.py`

```
"""Projekt Kaffeeshop, Stufe 2: eine abgeschlossene Bestellung."""

from dataclasses import dataclass
from decimal import Decimal

from warenkorb import Kunde, Position
from zahlungsarten import Zahlungsart


@dataclass
class Bestellung:
    nummer: int
    kunde: Kunde
    positionen: list[Position]
    zahlungsart: Zahlungsart

    def summe(self) -> Decimal:
        summe = Decimal(0)
        for position in self.positionen:
            summe += position.summe()
        return summe

    def ausgeben(self) -> None:
        print(f"Bestellung {self.nummer} von {self.kunde.name}")
        for position in self.positionen:
            prozent = int(position.produkt.mwst_satz() * 100)
            print(
                f"   {position.menge} x {position.produkt.name} "
                f"({prozent} % MwSt): {position.summe()} Euro"
            )
        print(f"   Summe: {self.summe()} Euro")
        print(f"   {self.zahlungsart.bezahlen(self.summe())}")
```

5.4 `main.py` für Stufe 2

Dazu kommen die Imports `from bestellung import Bestellung` und `from zahlungsarten import Kreditkarte, Lastschrift, Rechnungskauf, Zahlungsart`. Im Menü kommt die Zeile `7  Meine Bestellungen` dazu. In `main()` kommen `bestellungen: list[Bestellung] = []`, `case "6": bestellen(korb, bestellungen)` und `case "7": bestellungen_anzeigen(bestellungen)` dazu. Die Funktion `bestellen` wird ersetzt durch:

```
def zahlungsart_waehlen() -> Zahlungsart:
    while True:
        match input("Zahlung mit (1) Kreditkarte, (2) Lastschrift, (3) Rechnung: "):
            case "1":
                return Kreditkarte(input("Kartennummer: "))
            case "2":
                return Lastschrift(input("IBAN: "))
            case "3":
                return Rechnungskauf()
            case _:
                print("Bitte 1, 2 oder 3 eingeben.")


def bestellen(korb: Warenkorb, bestellungen: list[Bestellung]) -> None:
    if korb.ist_leer():
        print("Der Warenkorb ist leer. Es gibt nichts zu bestellen.")
        return
    zahlungsart = zahlungsart_waehlen()
    for position in korb.positionen:
        position.produkt.verkaufen(position.menge)  # Bestand verringern
    nummer = len(bestellungen) + 1
    bestellung = Bestellung(nummer, korb.kunde, korb.positionen, zahlungsart)
    bestellungen.append(bestellung)
    korb.positionen = []  # NICHT clear(): die Bestellung behält ihre Liste
    bestellung.ausgeben()


def bestellungen_anzeigen(bestellungen: list[Bestellung]) -> None:
    if len(bestellungen) == 0:
        print("Noch keine Bestellungen.")
    for bestellung in bestellungen:
        bestellung.ausgeben()
```

Die Falle von Folie 18, ganz langsam erklärt:

- `Bestellung(…, korb.positionen, …)` kopiert die Liste **nicht**. Die Bestellung bekommt nur einen zweiten Namen für **dieselbe** Liste.
- `korb.positionen.clear()` leert diese eine Liste. Dann ist auch die Bestellung leer, und Menüpunkt 7 zeigt nichts mehr.
- `korb.positionen = []` gibt dem Korb eine **neue** leere Liste. Die alte Liste gehört danach nur noch der Bestellung.

Kontrolle (Folie 19): 3 Kaffee + 1 French Press mit Kreditkarte `1234567812345678`:

```
Bestellung 1 von Anna Berger
   3 x Kaffee Hausmischung 1 kg (7 % MwSt): 41.70 Euro
   1 x French Press 1 l (19 % MwSt): 24.87 Euro
   Summe: 66.57 Euro
   66.57 Euro von Karte ...5678 abgebucht
```

Nach zwei Bestellungen zeigt Menüpunkt 7 beide Bestellungen, auch wenn der Warenkorb leer ist.

---

6. Stufe 3: Zwei Zusätze zum Aussuchen (Folie 21)

Versandkosten: 4,90 €, ab 50 € Warenwert versandfrei.

```
def versandkosten(warenwert: Decimal) -> Decimal:
    if warenwert >= 50:
```

```
        return Decimal(0)
    return Decimal("4.90")
```

Beispiele: 46,38 € kostet 4,90 € Versand, macht zusammen 51,28 €. Bei 66,57 € ist der Versand frei.

Rabattcode mit `match`: `DHBW10` gibt 10 % auf den Warenwert, `WILLKOMMEN5` gibt 5 € ab 25 € Warenwert.

```
def rabatt(code: str, warenwert: Decimal) -> Decimal:
    match code:
        case "DHBW10":
            return (warenwert * Decimal("0.10")).quantize(Decimal("0.01"))
        case "WILLKOMMEN5" if warenwert >= 25:
            return Decimal("5.00")
        case _:
            return Decimal(0)
```

Das `if` hinter dem `case` heißt **Guard**. Der Fall greift nur, wenn auch die Bedingung stimmt. Beispiele: `DHBW10` bei 66,57 € ergibt 6,66 € Rabatt. `WILLKOMMEN5` bei 20 € ergibt 0 €, bei 30 € ergibt es 5,00 €.

Sortieren ohne `sorted()` (Tipp der Folie): Tausch das günstigste Produkt nach vorn, dann das zweitgünstigste und so weiter. Das ist **Selection Sort** aus GIT Kapitel 8.

---

7. Erklären können: 10 Fragen, die morgen kommen könnten

1. **Warum steht `return None` in `finden` nach der Schleife und nicht im `else`?** Sonst endet die Methode schon beim ersten Produkt, das nicht passt.
2. **Was ist eine Hat-ein-Beziehung?** Ein Objekt hat ein anderes als Attribut, zum Beispiel `korb.kunde`. Das ist keine Vererbung.
3. **Warum gibt `hinzufuegen` ein `bool` zurück und druckt nichts?** Die Klasse rechnet nur. Was der Nutzer sieht, entscheidet `main.py`. So kann man die Klasse auch ohne Terminal testen, zum Beispiel mit pytest.
4. **Warum zählt die Menge im Korb mit?** Sonst könnte man 3 + 3 French Press einpacken, obwohl nur 3 da sind.
5. **Warum `bruttopreis` ohne Klammern?** Es ist eine Property, sie sieht aus wie ein Attribut.
6. **Was passiert bei `abc` als Artikelnummer?** `zahl_einlesen` fragt erneut, weil `"abc".isdigit()` `False` ist.
7. **Warum die Summe vor dem Leeren merken?** Nach `korb.positionen = []` ist die Gesamtsumme 0.
8. **`clear()` oder `= []`?** `= []`, sonst verliert die Bestellung ihre Positionen. Es sind zwei Namen für eine Liste.
9. **Was ist hier polymorph?** `mwst_satz()` und `bezahlen()`. Derselbe Aufruf macht je nach Klasse etwas anderes.
10. **Wozu `@dataclass`?** Es schreibt `__init__` (und `__repr__`) automatisch für reine Datenklassen wie `Kunde`.

---

8. Typische Fehler

- `return None` im `else` innerhalb der Schleife: Dann findet `finden` nur das erste Produkt.
- In `hinzufuegen` die Menge im Korb vergessen, dann kann man mehr einpacken, als auf Lager ist.
- Zweimal dasselbe Produkt als zwei Positionen anlegen statt die Menge zu erhöhen.
- `produkt.bruttopreis()` mit Klammern: `TypeError`, weil es eine Property ist.
- `int(input(...))` statt `zahl_einlesen(...)`: Das Programm stürzt bei `abc` ab.

- `korb.positionen.clear()` nach dem Anlegen der Bestellung: Die Bestellung ist danach leer.
- `summe = 0` statt `Decimal(0)`: Das läuft, aber `ty check` meldet „invalid-return-type“, weil die Summe dann `0` (int) oder `Decimal` sein kann.
- Den Bestand schon beim Hinzufügen verringern statt erst beim Bestellen.
- Import vergessen (`from lager import Lager`), das gibt einen `NameError`.

---

9. Plan für morgen

1. **Startpaket** aus Moodle laden, nach `prog1/kaffeeshop` entpacken, `uv run main.py` starten, Punkt 1 ausprobieren.
2. **`lager.py`** schreiben (Abschnitt 4.1). Danach Menüpunkte 1 und 2 einbauen und die Suche nach `kaffee` testen. Etwa 30 Minuten.
3. **`warenkorb.py`** schreiben (Abschnitt 4.2). Erst `Kunde` und `Position`, dann `Warenkorb`. Etwa 45 Minuten.
4. **`main.py`** fertig bauen (Abschnitt 4.3) und die Tabelle in Abschnitt 4.4 abarbeiten. Damit ist Stufe 1 fertig.
5. Am Nachmittag Stufe 2, Teil für Teil, und immer wieder `uv run ruff check .` und `uv run ty check` laufen lassen.

Wenn es hakt: Erst laut sagen, was die Zeile tun soll, dann den Partner fragen, dann den Dozenten. Er geht herum und hilft.